Shai-Hulud: una minaccia nella supply chain JavaScript

Gruppo WannaCry

s358012 - Amedeo Marino
s361615 - Andrea Zanzarella
s356980 - Andrei Daniel Stefan
s361797 - Davide Giordano
s361817 - Leonardo Miani



Indice

1 Introduzione 2
2 Descrizione Shai-Hulud 2
3 Analisi Tecnica 3
4 Casi Reali 4
5 Conseguenze e Impatti 5
6 Mitigazioni e Comportamenti Corretti 6
7 Conclusioni e Discussione 7
Bibliografia 8
8 Allegato Utilizzo LLM 10
8.a Amedeo Marino S358012 10
8.b Modello utilizzato 10
8.c Modalità di utilizzo 10
8.d Andrei Daniel Stefan (S356980) 11
8.e Modello utilizzato 11
8.f Modalità di utilizzo 11

1 Introduzione

Un software moderno è spesso costruito riutilizzando codice già scritto e testato in precedenza, sotto forma di librerie. Molte sono distribuite gratuitamente online su registri pubblici sotto forma di pacchetti. npm è una delle piattaforme di pacchetti per il linguaggio di programmazione JavaScript, nonché vettore di attacco del protagonista di questa relazione, il worm Shai-Hulud.

Un worm è un programma malevolo che ha la capacità di propagarsi autonomamente da un sistema all'altro. Shai-Hulud ricade sotto questa classificazione: tra il 14 e il 16 settembre 2025 [1] ha infettato centinaia di pacchetti npm, diffondendo pubblicamente dati privati delle vittime. Il nome richiama i giganteschi vermi del deserto del romanzo «Dune» di Frank Herbert. Il malware pubblicava infatti i dati rubati in un nuovo progetto GitHub (una piattaforma online dedicata allo sviluppo software) pubblico sotto il nome Shai-Hulud [2].

2 Descrizione Shai-Hulud

Non è ancora confermato come il primo pacchetto sia stato infettato: è possibile collegare la diffusione del worm ad una campagna di phishing avvenuta pochi giorni prima (8 settembre 2025), dove le e-mail dell'attaccante, impersonando npm, richiedevano di aggiornare dati relativi all'autenticazione a due fattori [3]. In questo modo gli attaccanti avrebbero avuto accesso ai profili di diversi sviluppatori, e la possibilità di diffondere le prime copie di Shai-Hulud.

Questo tipo di attacco è diretto contro la supply chain del software: non colpisce la vittima direttamente, ma un componente che si sceglie di usare di propria iniziativa, fidandosi quasi ciecamente. Quando uno sviluppatore installa un pacchetto infetto, il codice malevolo viene eseguito in fase di installazione, cercando sul computer credenziali per poterle rubare. Se Shai-Hulud trova anche un token npm, credenziali usate per pubblicare pacchetti a nome del proprietario, modifica i pacchetti della vittima ripubblicandoli infetti, accelerando la propria diffusione.

Le vittime di questo attacco sono tutti gli sviluppatori che utilizzano JavaScript, anche fuori da contesti professionali: il linguaggio è utilizzato ampiamente in ambienti accademici e hobbistici. Sono colpiti individui, aziende e sistemi di integrazione continua (CI), i server che compilano e pubblicano software in modo automatico. Shai-Hulud era progettato specificamente per Linux e macOS e non agiva su Windows [2], [4].

L'attacco non si basa su un'unica falla di sicurezza, ma su diverse debolezze del sistema. La prima è la fiducia implicita: un pacchetto scaricato da npm viene installato ed eseguito senza particolari controlli. Durante questi processi è possibile l'esecuzione di codice, cosa che il worm sfrutta per entrare in azione senza che lo sviluppatore se ne accorga [1]. A ciò si aggiunge il fatto che token e chiavi sono spesso facilmente raggiungibili sul computer dello sviluppatore e rubate con facilità. Infine mancano controlli sulla provenienza dei pacchetti: il ricercatore di sicurezza Nicholas Weaver osserva che la pubblicazione può avvenire in modo automatico tramite le credenziali npm, senza il consenso esplicito di un umano [2].

Tra gli asset in gioco, figurano:

- credenziali: token GitHub e npm, chiavi di accesso a cloud AWS, Google Cloud e Azure, e altre informazioni riservate lasciate nelle “variabili d’ambiente”, dati memorizzati dal sistema operativo dove spesso si conservano credenziali [4];
- progetti GitHub: ulteriore strumento di raccolta di credenziali e segreti [4];
- reputazione di aziende e progetti: compagnie come Crowdstrike, colpita anch’essa da Shai Hulud, possono subire danni di immagine traducibili in perdite di denaro enormi.

Secondo alcune fonti, il probabile attore dietro Shai-Hulud è TeamPCP, un gruppo cybercriminale spinto da motivazioni finanziarie. La stessa organizzazione è ritenuta responsabile delle successive ondate Shai-Hulud, e, nel maggio 2026,ha pubblicato il codice sorgente del worm [5].

La sua diffusione è stata bloccata nel giro di 3 giorni, grazie ad una serie di azioni: la pubblicazione online di elenchi dei pacchetti compromessi [4], la rimozione di questi dai registri npm [2] e il blocco del traffico verso l’indirizzo web verso cui Shai-Hulud inviava parte dei dati rubati. Da parte delle vittime, è stato necessario eliminare i pacchetti infetti installati localmente e sostituire tutte le chiavi di accesso [6].

3 Analisi Tecnica

Le ricostruzioni indicano come vettore iniziale una campagna di **phishing**: un’e-mail mascherata da avviso di sicurezza di npm, che spingeva i manutentori ad «aggiornare» le proprie opzioni di autenticazione a più fattori e a inserire le credenziali su un sito controllato dall’attaccante [3]. Con credenziali valide, l’attaccante può pubblicare nuove versioni di pacchetti legittimi,

Il worm funziona in modo analogo a un virus e ogni vittima diventa a sua volta un infettore:

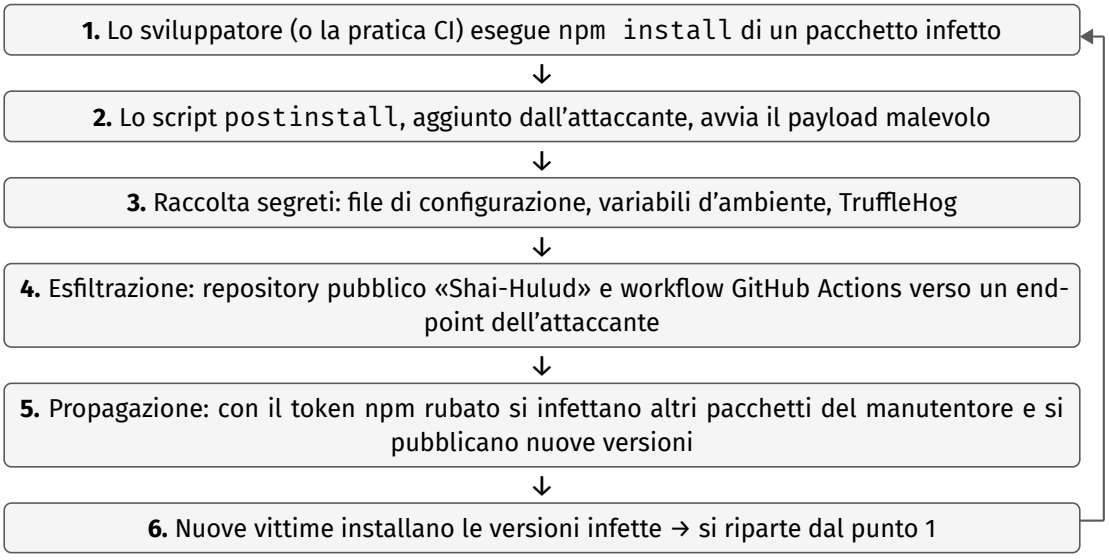


Figura 1: Ciclo di infezione e propagazione di Shai-Hulud

Nel dettaglio, si comporta così:

- **Esecuzione.** Il payload è un file javascript offuscato di circa 3,6 MB (*bundle.js*), lanciato dallo script **postinstall**. Il codice verifica di girare su Linux o macOS e salta i sistemi Windows [4], [7].
- **Raccolta di segreti.** Il malware legge le variabili d'ambiente (compreso GITHUB_TOKEN), i token personali GitHub e i token npm; interroga i gestori di segreti di AWS e Google Cloud e i servizi di metadata delle istanze cloud; scarica ed esegue TruffleHog, un legittimo strumento open source di ricerca di segreti, sull'intero file-system. L'uso di uno strumento lecito, presentato come una «scansione di sicurezza», e la gestione silenziosa degli errori riducono le tracce lasciate [4], [7].
- **Esfiltrazione.** Con il token GitHub trovato crea un repository pubblico chiamato Shai-Hulud nell'account della vittima, con un file JSON contenente informazioni di sistema, variabili d'ambiente e segreti. In più, aggiunge un branch con un workflow GitHub Actions (shai-hulud-workflow.yml) che, a ogni push, invia il contenuto di secrets a un endpoint webhook.site controllato dall'attaccante [2], [4].
- **Esposizione di repository privati.** Il worm enumera i repository accessibili alla vittima, anche privati, e li rende visibili pubblicamente: Aikido lo descrive come un cambio di visibilità, TrendAI come la copia in repository pubblici marcati «Shai-Hulud Migration» [8], [9].
- **Propagazione.** Se trova un token npm valido (in ~/.npmrc o nell'ambiente), lo verifica, elenca i pacchetti di cui l'utente è manutentore (fino a 20), incrementa la versione, aggiunge lo script postinstall e il file *bundle.js* e ripubblica. In questo modo l'infezione «salta» da un manutentore al successivo senza intervento umano [4], [7].

Debolezze sfruttate

L'analisi mostra che nessuna singola vulnerabilità software è all'origine dell'attacco: il worm sfrutta **debolezze strutturali e di processo**:

1. **Esecuzione automatica di script di installazione**: il gestore di pacchetti esegue codice di terze parti senza consenso esplicito.
2. **Token a lunga durata**: i token npm e GitHub rubati restano validi finché nessuno li revoca, permettendo la propagazione [6].
3. **Segreti in chiaro** nelle macchine di sviluppo (`.npmrc`, file `.env`, variabili d'ambiente), facilmente recuperabili da un processo qualsiasi dell'utente.
4. **Autenticazione debole dei manutentori**: l'MFA basata su codici temporanei (TOTP) può essere aggirata da un phishing ben fatto; GitHub ha annunciato il passaggio a metodi FIDO proprio per questo [10].
5. **Fiducia transitiva nelle dipendenze** e aggiornamenti automatici (intervalli di versione come `^1.2.3`) che portano subito sul sistema le versioni appena pubblicate.
6. **Pipeline CI/CD con permessi eccessivi**, che concentrano segreti di alto valore.

4 Casi Reali

Il primo episodio documentato riguarda `@ctrl/tinycolor`, una libreria usata da un numero molto elevato di progetti e scaricata milioni di volte a settimana. Alcune sue versioni sono state modificate con codice malevolo che si avviava automaticamente durante l'installazione [1]. Per chi installava la dipendenza non c'era nulla di anomalo da notare: il comando era quello abituale. È proprio questo che rende il caso istruttivo,

perché la fiducia verso una dipendenza molto diffusa è bastata a portare il codice malevolo dentro i sistemi. Il danno non era solo nella macchina infetta: il codice cercava credenziali e le inviava a server esterni o le pubblicava in repository GitHub pubblici [11]. Una credenziale esposta può quindi portare a nuovi accessi, ben oltre il singolo computer.

Nella stessa ondata sono stati colpiti anche pacchetti pubblicati da CrowdStrike, una delle aziende di cybersecurity più note [1]. Il fatto che un'azienda che si occupa di sicurezza sia finita nella catena suggerisce che l'attacco non dipendesse dalla competenza di chi pubblicava: ciò che contava era l'accesso all'account. Non abbiamo prove che l'attaccante abbia scelto CrowdStrike per il suo prestigio, ma il caso mostra che la reputazione di un editore non basta a proteggere chi scarica i suoi pacchetti.

Il numero più citato è quello dei pacchetti compromessi. La CISA parla di oltre 500 pacchetti nel suo alert del 23 settembre 2025 [6]. GitHub ha dichiarato di averne rimossi più di 500 e di aver bloccato il caricamento di nuove versioni che contenevano gli stessi indicatori di compromissione [12]. La risposta mostra la sequenza tipica di un incidente di questa scala: prima si eliminano i pacchetti infetti, poi si impedisce che tornino con le stesse tracce. Il numero, però, dice poco da solo: ciò che conta è che bastano poche ore perché un singolo punto d'ingresso raggiunga centinaia di progetti.

Nel novembre 2025 il worm è tornato in una forma più aggressiva, ribattezzata Shai-Hulud 2.0. I ricercatori di Datadog hanno contato circa 800 pacchetti npm compromessi e hanno osservato un cambiamento importante: il worm non si limitava più a eseguire codice durante l'installazione, ma installava sulla macchina della vittima un *runner* self-hosted di GitHub Actions, cioè un agente che permette di eseguire comandi da remoto [13]. In pratica, la macchina di uno sviluppatore, o un server di build, diventava una porta di accesso permanente per l'attaccante. Il bersaglio non è più solo chi installa un pacchetto, ma anche l'infrastruttura che produce il software distribuito agli utenti.

Nel 2026 il meccanismo è stato riutilizzato da altri attori. Il 1° giugno 2026 un pacchetto di 32 librerie pubblicate sotto il namespace `@redhat-cloud-services` è stato compromesso con una variante chiamata Miasma, che Red Hat identifica come una versione di Mini Shai-Hulud [14].

5 Conseguenze e Impatti

L'attacco ha mostrato come la compromissione di un pacchetto npm possa avere conseguenze ben oltre il singolo progetto che lo utilizza. Il meccanismo di propagazione automatica ha permesso al worm di raggiungere centinaia di pacchetti, sfruttando la fiducia degli sviluppatori nelle dipendenze software e negli aggiornamenti pubblicati dai manutentori [6]. Il caso mette quindi in evidenza un rischio tipico della software supply-chain: un componente compromesso può diventare il punto di ingresso per attaccare molti sistemi diversi.

Una delle conseguenze più immediate è stata l'esposizione di dati riservati. Il malware raccoglieva token di accesso, credenziali per servizi cloud e variabili d'ambiente, per poi pubblicarli in file JSON all'interno di repository GitHub pubblici [11]. In alcuni casi sono stati esposti anche repository originariamente privati, con il rischio di rendere

accessibili codice proprietario e informazioni aziendali [15]. Le credenziali sottratte, inoltre, potevano essere utilizzate per accedere ad altri servizi, ampliando gli effetti dell'infezione iniziale.

La variante Shai-Hulud 2.0 ha aggiunto un ulteriore rischio per gli ambienti di sviluppo e le pipeline di integrazione e distribuzione continua (CI/CD). Secondo l'analisi di Datadog, il malware poteva registrare sulla macchina compromessa un runner self-hosted di GitHub Actions, offrendo all'attaccante una modalità di esecuzione remota dei comandi [13]. Di conseguenza, non erano a rischio soltanto i dati conservati sul sistema, ma anche l'infrastruttura utilizzata per produrre e distribuire il software.

Per le organizzazioni coinvolte, la gestione dell'incidente ha richiesto attività di verifica dei sistemi, rimozione delle dipendenze compromesse e revoca o sostituzione delle credenziali potenzialmente esposte, come raccomandato anche dalla CISA [6]. Queste operazioni comportano un impiego di tempo e risorse, oltre a possibili interruzioni del normale lavoro di sviluppo. Va considerato anche l'effetto sulla fiducia nei pacchetti di terze parti: il coinvolgimento di una libreria diffusa come `@ctrl/tinycolor` e di pacchetti riconducibili a CrowdStrike dimostra che la notorietà di un progetto o del suo editore, da sola, non garantisce l'assenza di codice malevolo [1].

6 Mitigazioni e Comportamenti Corretti

Le azioni compiute da Shai-Hulud possono essere riassunte in due punti principali [13]:

1. **Esfiltrazione di credenziali**: il worm aggiunge al file `package.json` del pacchetto `npm` infettato due nuovi script per inizializzare ed eseguire il runtime JavaScript «Bun», in modo tale da eseguire il payload bypassando i controlli di `node.js`. Le credenziali vengono cercate sia nel filesystem locale che in cloud, e poi caricate in una repository pubblica su GitHub.
2. **Replicazione automatica**: se il worm individua un token `npm` valido salvato in locale (ad esempio nel file `.npmrc`), interroga l'API di `npm` per identificare fino a 100 pacchetti mantenuti dall'utente. Dopodiché scarica tali pacchetti, inietta i propri script di `preinstall` nel `package.json`, incrementa la versione patch e ripubblica automaticamente i pacchetti infetti tramite `npm publish`.

Una prima contromisura da adottare per prevenire l'attivazione del worm è aggiungere il flag `--ignore-scripts` durante l'installazione delle dipendenze con `npm install`, evitando così l'esecuzione dello script `preinstall` [16]. Sempre in ambito dipendenze, è opportuno cancellare la cache `npm` e modificare il `package.json` fissando le dipendenze a una versione nota rilasciata in data precedente all'attacco [15].

Per prevenire l'ulteriore propagazione del worm ed evitare la compromissione dei propri account, ciascun utente dovrebbe abilitare servizi di autenticazione a più fattori, idealmente tramite *chiavi di sicurezza hardware* (es. YubiKey). In questo modo, quando il malware tenta di completare la chiamata API per `npm publish`, la piattaforma rifiuta la richiesta in assenza di una firma crittografica forte o di un OTP hardware valido, bloccando sul nascere la catena di infezione della supply chain [17].

7 Conclusioni e Discussione

Il caso Shai-Hulud rende visibile il punto critico della supply chain del software, ovvero la fiducia. Secondo le ricostruzioni, bastano le credenziali di pochi sviluppatori, ottenute tramite phishing, per infettare centinaia di pacchetti. Ciò suggerisce che è ancora il fattore umano ad essere uno degli anelli deboli della catena. Senza un accesso iniziale Shai-Hulud non sarebbe riuscito a penetrare nell'ecosistema npm. Allo stesso tempo sono stati evidenziati errori di design nei processi gestiti da npm: l'upload di pacchetti senza conferma umana e l'esecuzione silenziosa di codice. Questi meccanismi, ancor prima dell'attacco, si prestano ad essere sfruttati per scopi illeciti, rendendo npm un bersaglio attraente.

Ne nasce un paradosso. Aggiornare le dipendenze serve a ricevere correzioni di sicurezza, ma la nuova versione può contenere essa stessa il malware. Un rimedio parziale è fissare i pacchetti ad una versione precisa, evitando l'installazione di nuove versioni al prezzo di restare indietro con le patch di sicurezza.

Una volta che il worm raggiunge un computer, la vittima è ignara di ciò che sta succedendo. Le credenziali sono facilmente rubate perché salvate in chiaro: qualsiasi processo dell'utente ha accesso a questi file, anche senza privilegi amministrativi. Le credenziali sono poi diffuse pubblicamente e utilizzate per propagare il worm.

In termini di rischio, l'asset più esposto sono le credenziali. Il probabile attore dietro Shai-Hulud, TeamPCP, è mosso da motivazioni economiche. Le vulnerabilità sono strutturali e non tecniche: fiducia implicita, token di lunga durata, segreti in chiaro. L'impatto va dall'esposizione di repository privati alla compromissione delle pipeline di sviluppo. Tra le mitigazioni citate, l'autenticazione a più fattori con chiavi hardware e l'opzione `--ignore-scripts` riducono il rischio, ma nessuna basta da sola.

La prima ondata è stata fermata in pochi giorni, ma l'attacco è tornato in forme più gravi (Shai-Hulud 2.0, nel novembre 2025) e il codice sorgente del worm è stato pubblicato nel maggio 2026, il che rende più probabili nuove varianti. Chiunque usi npm (professionisti, hobbisti e accademici) dovrebbe quindi trattare le proprie credenziali come un bersaglio. Va infine ricordato che alcune parti dell'analisi, come l'origine del primo attacco e l'attribuzione, restano non confermate.

Bibliografia

[1] B. O. Pandya Kush, «Updated and Ongoing Supply Chain Attack Targets CrowdStrike npm Packages», *Socket.dev*, set. 2025, Consultato: 15 ottobre 2026. [Online]. Disponibile su: https://socket.dev/blog/ongoing-supply-chain-attack-targets-crowdstrike-npm-packages

[2] B. Krebs, «Self-Replicating Worm Hits 180+ Software Packages». Consultato: 17 ottobre 2026. [Online]. Disponibile su: https://krebsonsecurity.com/2025/09/self-replicating-worm-hits-180-software-packages/

[3] B. Krebs, «18 Popular Code Packages Hacked, Rigged to Steal Crypto». Consultato: 17 ottobre 2026. [Online]. Disponibile su: https://krebsonsecurity.com/2025/09/18-popular-code-packages-hacked-rigged-to-steal-crypto/

[4] A. Kurmi, «Shai-Hulud: Self-Replicating Worm Compromises 500+ NPM Packages». Consultato: 17 ottobre 2026. [Online]. Disponibile su: https://www.stepsecurity.io/blog/ctrl-tinycolor-and-40-npm-packages-compromised#acknowledgements

[5] A. Culafi, «Shai-Hulud Hackers TeamPCP: Lucky or Skilled?». Consultato: 17 ottobre 2026. [Online]. Disponibile su: https://www.darkreading.com/application-security/shai-hulud-hackers-teampcp-lucky-skilled

[6] CISA, «Widespread Supply Chain Compromise Impacting npm Ecosystem», set. 2025, Consultato: 15 ottobre 2026. [Online]. Disponibile su: https://www.cisa.gov/news-events/alerts/2025/09/23/widespread-supply-chain-compromise-impacting-npm-ecosystem

[7] J. Moore, «Shai-Hulud Worm Compromises npm Ecosystem in Supply Chain Attack». Consultato: 15 ottobre 2026. [Online]. Disponibile su: https://unit42.paloaltonetworks.com/npm-supply-chain-attack/

[8] «s1ngularity/Nx attackers strike again». Consultato: 14 ottobre 2026. [Online]. Disponibile su: https://www.aikido.dev/blog/s1ngularity-nx-attackers-strike-again

[9] «What We Know About the NPM Supply Chain Attack». Consultato: 14 ottobre 2026. [Online]. Disponibile su: https://www.trendaisecurity.com/en-us/resources-insights/trendai-security-blog/npm-supply-chain-attack

[10] «Our plan for a more secure npm supply chain». Consultato: 17 ottobre 2026. [Online]. Disponibile su: https://github.blog/security/supply-chain-security/our-plan-for-a-more-secure-npm-supply-chain/

[11] Cohesity REDlab, «Supply Chain Compromise: Shai-Hulud exploits npm ecosystem». Consultato: 15 ottobre 2026. [Online]. Disponibile su: https://www.cohesity.com/platform/redlab/advisories/supply-chain-compromise-shai-hulud/

[12] J. Barron, «GitHub details upcoming changes to improve security in wake of Shai-Hulud worm in npm ecosystem», *SD Times*, set. 2025, Consultato: 15 ottobre 2026. [Online]. Disponibile su: https://sdtimes.com/security/github-details-upcoming-changes-to-improve-security-in-wake-of-shai-hulud-worm-in-npm-ecosystem/

[13] Datadog Security Labs, «The Shai-Hulud 2.0 npm worm: analysis, and what you need to know». Consultato: 15 ottobre 2026. [Online]. Disponibile su: https://securitylabs.datadoghq.com/articles/shai-hulud-2.0-npm-worm/

[14] Red Hat Customer Portal, «RHSB-2026-006 Supply chain compromise of @redhat-cloud-services npm packages», lug. 2026, Consultato: 17 ottobre 2026. [Online]. Disponibile su: https://access.redhat.com/security/vulnerabilities/RHSB-2026-006

[15] Wiz Research, «Shai-Hulud 2.0 Supply Chain Attack: 25K+ Repos Exposing Secrets». Consultato: 17 ottobre 2026. [Online]. Disponibile su: https://www.wiz.io/blog/shai-hulud-2-0-ongoing-supply-chain-attack

[16] Liran Tal, «NPM Security Best Practices: How to Protect Your Packages After the 2025 Shai Hulud Attack». Consultato: 17 ottobre 2026. [Online]. Disponibile su: https://snyk.io/articles/npm-security-best-practices-shai-hulud-attack/

[17] «Requiring 2FA for package publishing and settings modification». Consultato: 17 ottobre 2026. [Online]. Disponibile su: https://docs.npmjs.com/requiring-2fa-for-package-publishing-and-settings-modification

8 Allegato Utilizzo LLM

8.a Amedeo Marino S358012

Autore delle sezioni:

- introduzione;
- descrizione;
- conclusione.

8.b Modello utilizzato

Claude Sonnet 5.5 High

8.c Modalità di utilizzo

I paragrafi in questione sono stati innanzitutto scritti senza aiuti esterni, eccetto l'utilizzo delle fonti dichiarate in bibliografia. Il modello LLM è stato utilizzato per verificare l'aderenza a tutti i requisiti stabiliti nelle istruzioni.

Il modello ha avuto come contesto i seguenti file:

- `2026_DidatticaAttiva Relazione1`;
- `relazione esempio_1`;
- `relazione esempio_2`;
- `relazione esempio_3`.

Il prompt principale, come inizio del dialogo, è stato il seguente:

```
Sto scrivendo la prima relazione del corso di sicurezza dei sistemi
informativi. Mi occupo in particolare della parte di introduzione e
descrizione non tecnica del worm Shai Hulud. Potresti controllare e
suggerire eventuali miglioramenti in base alla consegna data dal
professore e alle relazioni di esempio?

[ paragrafi nella loro forma originale ]

Esplora anche queste pagine web che ho usato come fonti, controllando che
tutti i dati riportati siano coerenti:
https://www.darkreading.com/application-security/shai-hulud-hackers-
teampcp-lucky-skilled
https://krebsonsecurity.com/2025/09/18-popular-code-packages-hacked-
rigged-to-steal-crypto/
https://krebsonsecurity.com/2025/09/self-replicating-worm-hits-180-
software-packages/
https://www.stepsecurity.io/blog/ctrl-tinycolor-and-40-npm-packages-
compromised
https://socket.dev/blog/ongoing-supply-chain-attack-targets-crowdstrike-
npm-packages
```

Il modello ha fatto notare come il testo fosse poco adatto a un lettore non tecnico. Inoltre, c'erano alcune discrepanze tra le dichiarazioni fatte dall'autore e quelle delle fonti effettive, e l'aderenza non era completa al modello del Cyber Risk Management.

Le modifiche risultanti hanno portato a una pesante riscrittura dell'insieme, aggiungendo definizioni e precisazioni per un pubblico non tecnico. Anche il discorso è stato rimodellato, portando più informazioni dall'introduzione alla descrizione e descrivendo gli avvenimenti in un ordine meno confusionario.

Questa porzione di relazione è stata terminata tramite un secondo prompt, per la verifica di errori grammaticali, logici, chiarezza dell'esposizione e altri difetti come la presenza di doppi spazi:

```
[ testo paragrafi ]

Puoi controllare queste parti della relazione? Verifica il testo sia
chiaro da leggere, non ci siano frasi troppo lunghe, o errori
grammaticali/sintattici/logici
```

La risposta è stata usata per correggere tutti gli errori grammaticali e la riformulazione di alcune frasi. Il contenuto è rimasto invariato.

8.d Andrei Daniel Stefan (S356980)

Autore delle sezioni:

- casi reali

8.e Modello utilizzato

Claude Sonnet 5.5 Medium

8.f Modalità di utilizzo

Il primo prompt è stato il seguente:

```
Chi era il paziente 0 dell'attacco shai hulud npm nel settembre 2025? come
è riuscito a entrare nel pc della vittima e a iniziare a diffondersi dato
che è un worm autoreplicante?
```

Il modello ha risposto con delle informazioni molto generiche. Il successivo prompt è stato inviato allegando il documento `2026_DidatticaAttiva Relazione1.pdf`

```
Devo scrivere una relazione riguardante questo attacco seguendo le
istruzioni seguenti. in particolare devo scrivere la parte dei casi reali
ed esempi noti:
```

Il suo output è stato, anche qui, molto generico, non riuscendo a comprendere che io dovessi concentrarmi solo sulla sezione dei casi reali e, inoltre, dandomi delle fonti che, controllando, non mi sono sembrate molto affidabili (probabilmente per la loro poca fama online).

Successivi prompt hanno aiutato a raffinare la ricerca e a concentrarsi sull'argomento richiesto, offrendo delle fonti con una notorietà maggiore.

Esempi di prompt successivi simili:

```
ma io voglio solo il paragrafo dei casi reali...non tutta l'analisi
dell'attacco che verrà fatta da qualcun altro, non da me

[risposta]

riscrivi. la descrizione di cosa sia shai hulud gia sarà in un paragrafo
descrittivo a parte.... devo parlare di casi reali dell'attacco shai
hulud!

[risposta]
```

L’output finale è stato letto, modificato e le fonti sono state controllate per verificarne l’autenticità e la veridicità dei fatti riportati.